MASTER 1 COMPUTER SCIENCE | MINI-PROJECT 2026–2027



FULL PROJECT REPORT

Precision Agriculture Monitoring

Using Machine Learning

DATA • PREPROCESSING • EXPLORATION • MODELING

PREPARED BY

BOUTRIA Manal
BOUAKKAZ Madjeda
AIT HOCINE Khadija
KHEDRI Manel

Contents

SECTION 1 **Introduction** 4

1.1 Context 4

1.2 Motivation 4

1.3 Project objective 4

1.4 Preprocessing objectives 5

1.5 Organisation of this report 5

SECTION 2 **Problem statement** 6

2.1 Task definition 6

2.2 Main question 6

2.3 Secondary questions 6

2.4 Scope and working assumptions 6

PART 1 **Dataset audit and preprocessing** 7

1. Role and deliverables of Student 1 7

2. Architecture (Student 1 part: imputation and encoding) 7

3. Dataset and features 7

4. Experimental protocol set up by Student 1 10

5. Limitations (data-related) 11

PART 2 **Univariate statistics, outliers and scaling** 12

1. Scope and data used 12

2. Data quality 12

3. Univariate statistics 12

4. Visualizations 14

5. Outlier detection 14

6. Target leakage affecting the analysis 15

7. Decisions and conclusions 15

8. Limitations 16

9. Deliverables 16

10. Objective and components 16

11. Experimental protocol 17

12. Results 18

13. Interpretation and decisions 19

14. Observation on the target (decision pending) 20

15. Limitations of Week 2 20

16. Reproduction and deliverables 21

PART 3 **Time-series exploration and decision gate** 22

1. Scope and data used 22
2. Timestamp quality 22
3. Reliability of sensor, zone and position metadata 23
4. Temporal dependence and stationarity 23
5. Daily and seasonal patterns 24
6. Target: drift and daily cycle 25
7. Cross-correlation and lead-lag relationships 26
8. Decision gate: is temporal cross-validation appropriate? 27
9. Temporal feature engineering 27
10. Splitting protocol 28
11. Limitations 29
12. Reproducibility 29

PART 4 **Feature analysis, selection and leakage control** 30
1. Data and splits 30
2. Target leakage 31
3. Feature analysis 31
4. Feature selection 34
5. Class imbalance 35
6. Robustness on the time-series folds 37
7. Leakage control and tests 37
8. Deliverables and how to use them 37
9. Decisions pending and limitations 37

SECTION 1

REPORT STRUCTURE

Introduction

Context, motivation, project objective and preprocessing objectives

1.1 Context

Precision agriculture relies on networks of IoT sensors that continuously measure the field (soil moisture and temperature, air temperature, humidity, radiation, nutrients, irrigation) so that water, fertiliser and protection measures can be applied where and when they are needed. Machine learning turns these measurements into decisions, for example by classifying the current environmental condition of a crop zone.

This project uses the *Precision Agriculture Monitoring Dataset* (Kaggle): 15,000 records from 30 sensor nodes, one record every 30 minutes from 2024-01-01 to 2024-11-08, with 32 columns covering IoT status, crop, spatial, environmental, soil, atmospheric and irrigation variables. The target `environmental_condition` has five classes: Optimal (42.55%), Dry_Stress (23.71%), Adverse (22.41%), High_Humidity (7.29%) and Heat_Stress (4.03%).

1.2 Motivation

A high score on this dataset is easy to obtain and easy to misread. The exploration carried out in this phase revealed five pitfalls that make a careful preprocessing and validation set-up necessary:

- **Target leakage.** `irrigation_status` is *On* in all Dry_Stress rows and in no other row (Cramér's V = 1.00); the irrigation amount and water flow are non-zero only in those same rows. A model using them would score perfectly and be useless in practice.
- **A target that behaves like a rule.** Dry_Stress coincides exactly with `soil_moisture_pct` ≤ 35, and a random forest reaches a macro F1 of about 0.996. Differences between models and between preprocessing choices are therefore hard to measure.
- **Strong class imbalance.** The ratio between the largest and the smallest class is 10.55 : 1, above the 1 : 3 threshold of the specification, so accuracy is misleading and macro F1, balanced accuracy and per-class recall are used.
- **Time structure and unreliable metadata.** The series is perfectly regular with a strong 24-hour cycle, whereas sensor, zone and position metadata do not describe stable physical locations. This rules out spatial cross-validation and raises the question of whether random folds are enough.
- **Likely synthetic data.** Many variables are truncated at fixed round bounds, so conclusions may not transfer to real deployments.

Without a leak-free pipeline, scalers, clipping bounds or selectors fitted on the whole dataset would let validation rows influence training, and the reported scores would be optimistic.

1.3 Project objective

The overall objective of the project is to build, evaluate and compare machine learning models that classify `environmental_condition` from simultaneous sensor readings, following a rigorous experimental protocol (repeated stratified cross-validation, statistical comparison, ablation and error analysis).

This report covers **Phase 1: exploration and leak-free preprocessing**. Its objective is to deliver to all groups a common, documented and reproducible base: audited data, a shared unfitted pipeline, fixed splits and justified preprocessing decisions. Model training, statistical tests and error analysis belong to the next phases.

1.4 Preprocessing objectives

#	Objective	What it requires	Part
O1	Audit data quality	Missing values, duplicates, units, timestamp regularity, plausible ranges	1, 2, 3
O2	Remove target leakage	Detect variables derived from the target and drop them once, as a fixed rule	1, 4
O3	Build a leak-free pipeline	Everything learned from data (imputation, clipping, scaling, encoding, selection, resampling) is fitted on the training part of each fold only	1, 2, 4
O4	Treat outliers and scale the features	Decide on clipping and scaling by measurement, with scaling optional for tree models	2
O5	Handle class imbalance	Quantify it and compare rebalancing strategies inside the training folds	1, 4
O6	Select informative features	Analyse relevance, redundancy and separability, and compare selection methods	4
O7	Define the validation protocol	Locked test set, 24-hour gap, 30 stratified folds, 5 temporal folds, temporal features	3, 4
O8	Guarantee reproducibility	`random_state = 42`, saved split indices, automatic leakage tests	3, 4

1.5 Organisation of this report

Part	Author	Content	Mandatory report sections covered
1	Student 1	Dataset audit, leakage check, class imbalance, imputation and encoding	5. Architecture, 6. Dataset and Features, 7. Experimental Protocol
2	Student 2	Univariate statistics, outlier detection, outlier handling and scaling	5. Architecture, 6. Dataset and Features
3	Student 3	Time-series audit, temporal features, split generation	5. Architecture, 6. Dataset and Features, 7. Experimental Protocol, 14. Limitations
4	Student 4	Feature analysis and selection, rebalancing, leakage control and tests	5. Architecture, 6. Dataset and Features, 14. Limitations

SECTION 2

REPORT STRUCTURE

Problem statement

Main question and secondary questions RQ1–RQ4

2.1 Task definition

Given the simultaneous sensor readings of one record (after removal of the leakage columns), predict which of the five classes of `environmental_condition` applies: Optimal, Dry_Stress, Adverse, High_Humidity or Heat_Stress. This is a multi-class classification of the *current state* of the field, not a forecasting task: the temporal analysis shows that no variable precedes another or the target, and all relations are at lag 0.

Models are judged by macro F1 (main metric), balanced accuracy and per-class recall, because 43% of the rows are Optimal and accuracy would reward a majority-class guess.

2.2 Main question

> How can the Precision Agriculture Monitoring data be explored and preprocessed, without target leakage and without letting validation or test rows influence training, so that machine learning models can be trained and compared fairly on `environmental_condition`?

2.3 Secondary questions

RQ	Question	Addressed in
RQ1	**Data quality and leakage.** Which variables are derived from the target or unreliable (irrigation variables, soil moisture, sensor, zone and position metadata), and how should they be handled?	Parts 1, 2, 3 and 4
RQ2	**Preprocessing choices.** Do outlier treatment, scaling and feature selection change model performance, and for which kinds of models (scale-sensitive or tree-based)?	Parts 2 and 4
RQ3	**Class imbalance.** How severe is the imbalance and which rebalancing strategy (class weights, SMOTE, ADASYN, undersampling) helps, for linear models and for tree ensembles?	Parts 1 and 4
RQ4	**Validation protocol.** Are random stratified folds sufficient given the strong temporal dependence, and do temporal features or temporal folds change the conclusions?	Parts 3 and 4

2.4 Scope and working assumptions

- The locked test set (the last 3,000 records, after a 24-hour gap) is used once, at the very end.
- Fixed rules, such as dropping the leakage columns, are applied once to the whole dataset; every transformation that learns parameters from data is fitted on the training part of each fold inside one pipeline.
- Spatial cross-validation is not used because sensor, zone and position metadata do not describe stable locations.
- All stochastic operations use `random_state = 42`.
- `soil_moisture_pct` is kept for now although it nearly defines Dry_Stress; its removal is left to the instructor's decision.
- Optimised models, hyperparameter tuning and statistical tests are outside the scope of this phase.

PART 1

STUDENT 1

Dataset audit and preprocessing

Audit, leakage check, class imbalance and preprocessing set-up

1. Role and deliverables of Student 1

Task assigned (Week 1 / Week 2)	Status	Deliverable
Dataset audit (source, license, units, missing values, duplicates)	Done	Exploratory_Data.csv
Target and leakage check	Done	eda_data.py
Class imbalance analysis	Done	imbalance.py, class_pie.png
Imputation and encoding (leak-free pipeline step)	Done	preprocessing.py
README.md and requirements.txt	Done	Requirement.txt
Cleaned dataset (leakage columns removed)	Done	data_clean.csv

2. Architecture (Student 1 part: imputation and encoding)

All learned transformations are encapsulated in an unfitted scikit-learn `Pipeline` that is fitted on the training fold only; the validation and test data are only transformed. The only operations applied to the whole dataset are structural ones that learn no parameter from the data (removing leakage columns, type fixing, duplicate check).

Step	Method	Justification
Numeric imputation	SimpleImputer(strategy='median')	Robust to outliers. The dataset currently has 0 missing values, so the imputer acts as a safeguard for new data.
Categorical imputation	SimpleImputer(strategy='most_frequent')	Same safeguard for crop_type, growth_stage, sensor_status.
Encoding	OneHotEncoder(handle_unknown='ignore')	All categorical features have fewer than 10 modalities (5, 5 and 3).
Excluded from features	timestamp, sensor_node_id, field_zone	Because they are identifiers

Scaling, outlier handling, feature selection and rebalancing are implemented by the other students in the same pipeline, and each step is optional through a parameter so that models that do not need it (tree-based models) can disable it without editing code.

3. Dataset and features

3.1 Source and license

Item	Value
Name	Precision Agriculture Monitoring Dataset — Distributed Sensor Data for Smart Farming

Item	Value
Platform / author	Kaggle, user colabsss
URL	https://www.kaggle.com/datasets/colabsss/precision-agriculture-monitoring-dataset
License	The dataset is listed under the CC0 (Public Domain) license, which generally allows reuse without copyright restrictions. The license information should be verified on the original Kaggle dataset page.
Collection period	2024-01-01 00:00 to 2024-11-08 11:30
Size	15,000 rows × 32 columns (31 features + 1 target)
Nature of the data	IoT sensor measurements (perfectly regular sampling, labels that follow thresholds).

3.2 Variables and units

Family	Variables (unit)
Temporal	timestamp
IoT	sensor_node_id (30 nodes); battery_level_pct (%); signal_strength_dbm (dBm); sensor_status (Active, Low_Battery, Weak_Signal)
Crop	crop_type (5: Tomato, Cotton, Maize, Wheat, Rice); growth_stage (5: Seedling, Vegetative, Flowering, Fruiting, Maturity)
Spatial	field_zone (5 zones); latitude, longitude (°)
Environmental	air_temperature_c (°C); relative_humidity_pct (%); wind_speed_m_s (m/s); wind_direction_deg (°); rainfall_mm (mm); leaf_wetness_pct (%); evapotranspiration_mm (mm)
Soil	soil_temperature_c (°C); soil_moisture_pct (%); soil_ph; soil_ec_ds_m (dS/m); nitrogen, phosphorus, potassium (mg/kg)
Atmospheric	atmospheric_pressure_hpa (hPa); solar_radiation_w_m2 (W/m²); light_intensity_lux (lux); co2_ppm (ppm)
Irrigation	irrigation_status (On/Off); irrigation_amount_l (L); water_flow_rate_l_min (L/min)
Target	environmental_condition — 5 classes (multi-class)

Types: 24 numeric, 6 categorical (including the target), 1 identifier-like column (sensor_node_id) and 1 datetime column.

3.3 Data quality audit

- **Missing values:** 0 in every column (0.00%), so no imputation strategy needs to be justified by missingness; imputation is kept as a safeguard only.
- **Duplicates:** 0 fully duplicated rows and 0 duplicates on the key (timestamp, sensor_node_id, field_zone). In fact all 15,000 timestamps are unique.
- **Timestamps:** perfectly regular 30-minute step (14,999 intervals of exactly 30 min, no gaps).
- **Independence of observations:** each timestamp holds one record, and consecutive records come from different sensors; no sensor provides a continuous series, so observations can be treated as independent rows (to be discussed with Student 3 for the temporal analysis).
- **Units:** given in the column names; values are physically plausible in the sample inspected (e.g. solar radiation is 0 at night). Detailed per-variable statistics are in Exploratory_Data.csv.

```python
import pandas as pd
import numpy as np
# Read the dataset:
df=pd.read_csv("data.csv")
print("Number of rows and columns in our dataset is: \t",df.shape)
print("Number of duplicates in our dataset is: \t",df.duplicated().sum())
print("Number of duplicates in key values in our dataset is: \t",df.duplicated(subset=["timestamp","sensor_node_id","field_zone"]).sum())
print("Number ofmissing values  in our dataset is: \t",df.isnull().sum().sum())
print(df.head(1))
```

```
[5]  ✓ 0.4s                                                                    Python

Number of rows and columns in our dataset is:    (15000, 32)
Number of duplicates in our dataset is:          0
Number of duplicates in key values in our dataset is:    0
Number ofmissing values  in our dataset is:      0
             timestamp sensor_node_id field_zone   latitude  longitude  \
0  2024-01-01 00:00:00         SN_007     Zone_C  11.696902  78.726563

  crop_type growth_stage  air_temperature_c  relative_humidity_pct  \
0    Tomato     Seedling              22.68                  87.98

   soil_temperature_c  ...  leaf_wetness_pct  co2_ppm  evapotranspiration_mm  \
0                21.2  ...             62.35   418.77                   0.62

   irrigation_status  irrigation_amount_l  water_flow_rate_l_min  \
0                Off                  0.0                    0.0

   battery_level_pct  signal_strength_dbm  sensor_status  \
0              69.23               -86.89         Active

   environmental_condition
0           High_Humidity

[1 rows x 32 columns]
```

3.4 Target variable and class imbalance

Class	Count	Percentage
Optimal	6,383	42.55%
Dry_Stress	3,557	23.71%
Adverse	3,361	22.41%
High_Humidity	1,094	7.29%
Heat_Stress	605	4.03%

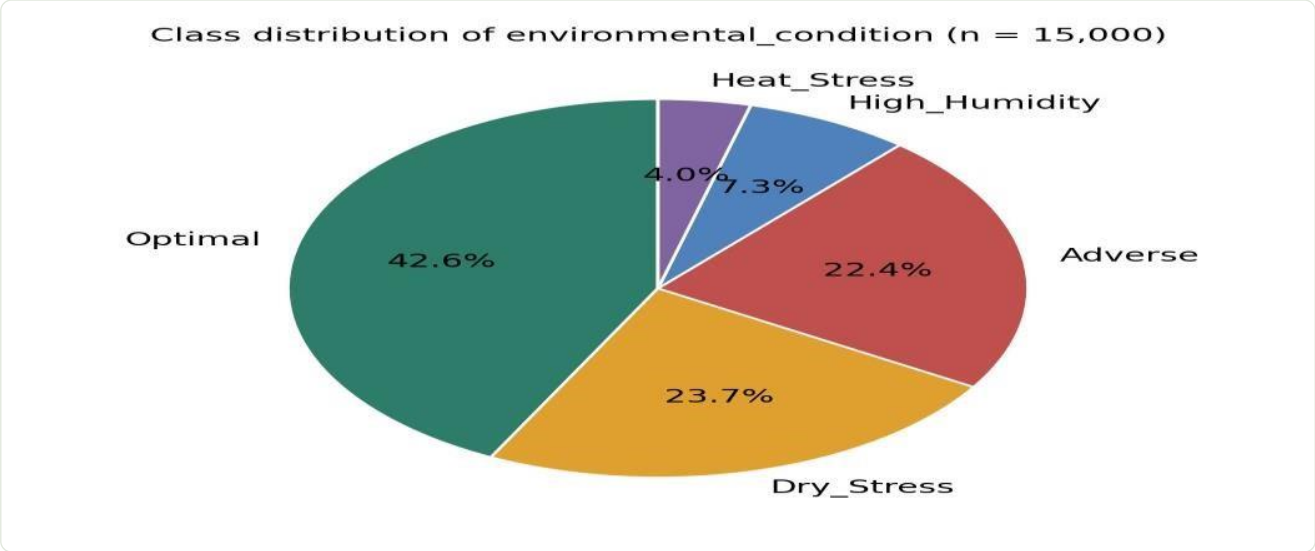


Figure 1. Class proportions (descriptive statistics on the whole dataset, which is allowed because nothing is learned from them).

Verification of Embalance classes to target variable:

```
[3]
TARGET='environmental_condition'
df= pd.read_csv("data.csv")
#Count the number of each class:
counts = df[TARGET].value_counts()
pct=(100*counts/len(df)).round(2)
table=pd.DataFrame({"count":counts,"percent":pct})
print(table)

# calculate max:min & compaire it with 1:3 :
ratio=counts.max()/counts.min()
print(f"imbalance ratio (max:min)={ratio:.2f}:1")
print("REAL dimbalance (ratio > 3):",ratio > 3)
```

```
...                          count  percent
environmental_condition
Optimal                       6383    42.55
Dry_Stress                    3557    23.71
Adverse                       3361    22.41
High_Humidity                 1094     7.29
Heat_Stress                    605     4.03
imbalance ratio (max:min)=10.55:1
REAL dimbalance (ratio > 3): True
```

- **Imbalance ratio (max:min) = 6383 / 605 = 10.55 : 1**, above the 1:3 threshold of the specification; normalized entropy = 0.845. The imbalance is therefore real and rebalancing will be evaluated.
- **Rebalancing** (class weights, SMOTE, random undersampling) is measured here but applied only inside the training folds of the pipeline (Student 4 compares the options).

3.5 Target leakage check

Following the specification (“verify that the target is not derived from an input variable”), I tested each variable's association with the target (ANOVA / chi-square) and trained a depth-2 decision tree on each single variable.

Variable	Evidence	Decision
irrigation_status	Value On occurs in all 3,557 Dry_Stress rows and in no other row (chi² = 15,000; Cramér's V = 1.00)	**Removed**
irrigation_amount_l	Equals 0 whenever irrigation is Off; ANOVA F = 14,117, η^2 = 0.79	**Removed**
water_flow_rate_l_min	Equals 0 whenever irrigation is Off; ANOVA F = 14,607, η^2 = 0.80	**Removed**

4. Experimental protocol set up by Student 1

Element	Choice
Structural cleaning	Applied once on the whole dataset: removal of leakage columns, duplicate check, type fixing. Output: data_clean.csv
Test set	20% stratified, isolated once (random_state = 42), locked, used a single time at the very end

Element	Choice
Cross-validation	Stratified 10-fold × 3 repetitions = 30 splits on the development set (80%); index files saved and shared (generated by Student 3)
Pipeline	Unfitted, delivered without model; each group appends its model and fits it on each training fold

5. Limitations (data-related)

- The data look synthetic and the labels appear rule-derived, so high scores may reflect recovery of the generating rules rather than agronomic generalization.
- Strong imbalance (10.55:1) makes the smallest class (Heat_Stress, 4%) the most fragile; results must be read with macro-F1 and per-class recall.

PART 2

STUDENT 2

Univariate statistics, outliers and scaling

Week 1 outlier detection · Week 2 outlier handling and scaling

PART I · WEEK 1 — UNIVARIATE STATISTICS AND OUTLIERS

1. Scope and data used

This section covers Student 2's Week 1 tasks: univariate statistics, visualizations, missing-value check, and outlier detection (IQR, z-score, Isolation Forest, LOF), plus a physical plausibility check.

The dataset has 15,000 rows and 32 columns, with one row every 30 minutes from 2024-01-01 to 2024-11-08. The target is `environmental_condition` (5 classes). All figures below are computed on the development set only: the first 11,952 rows chronologically (2024-01-01 to 2024-09-05). The last 3,000 rows are the locked test set and were not used. A 48-row (24-hour) gap separates the two sets to limit temporal leakage; those rows belong to neither. The split indices come from Student 3 (`dev_index.csv` , `test_index.csv`); the script checks that the two sets do not overlap. Exploration is descriptive. No statistic computed here is used to transform data; every data-dependent preprocessing parameter is fitted on training folds only in Week 2.

Two columns were removed from the analysis: latitude and longitude (location, not sensor measurements). 22 numeric sensor variables were analysed, and 20 remain after the leakage decision in Section 6.

2. Data quality

Check	Result (development set)
Rows × columns	11,952 × 32
Missing cells	0
Fully duplicated rows	0
Duplicate keys (timestamp, node, zone)	0
Physically impossible values	0 in every variable

The physical limits used were: pH 0–14; humidity, soil moisture, leaf wetness and battery 0–100 %; air temperature −50 to 60 °C; soil temperature −20 to 70 °C; pressure 300–1100 hPa; non-negative values for nutrients, rainfall, evapotranspiration and irrigation; and plausible bounds for radiation, light, wind, CO2 and signal strength.

3. Univariate statistics

Statistics for the 22 numeric variables (full table with Q1, median, Q3, kurtosis and z-score outliers in `Exploratory_Data_S2_univariate.csv`).

Variable	Min	Max	Mean	Std	Skewness	IQR outliers %	Zeros %
air_temperature_c	15.00	37.49	26.99	4.65	−0.02	0.00	0
relative_humidity_pct	30.00	98.00	68.10	12.59	0.00	0.00	0
soil_temperature_c	14.00	37.08	24.48	4.82	−0.01	0.00	0
soil_moisture_pct	10.00	95.00	45.91	14.93	0.04	0.33	0
soil_ph	4.50	8.50	6.51	0.70	−0.04	0.81	0
soil_ec_ds_m	0.20	3.39	1.50	0.50	0.09	0.42	0
nitrogen_mg_kg	15.00	150.00	70.38	24.59	0.07	0.33	0
phosphorus_mg_kg	10.00	100.00	45.09	14.82	0.06	0.44	0
potassium_mg_kg	20.00	160.00	80.04	24.70	0.03	0.40	0
atmospheric_pressure_hpa	982.94	1030.00	1009.93	6.97	−0.01	0.70	0
solar_radiation_w_m2	0.00	949.23	229.20	288.14	0.82	0.00	54.2
light_intensity_lux	0	100,000	22,833	29,005	0.88	0.00	54.2
wind_speed_m_s	0.01	15.00	2.98	2.11	1.39	3.23	0
wind_direction_deg	0.01	359.98	179.70	103.92	0.00	0.00	0
rainfall_mm	0.00	38.99	1.18	3.58	3.90	14.73	85.3
leaf_wetness_pct	0.18	98.45	42.43	11.88	0.17	0.81	0
co2_ppm	300.00	600.34	419.78	45.03	0.01	0.32	0
evapotranspiration_mm	0.00	5.37	1.97	1.34	0.54	0.00	0
irrigation_amount_l	0.00	79.99	10.48	21.28	1.89	23.48	76.5
water_flow_rate_l_min	0.00	15.00	1.99	4.02	1.87	23.48	76.5
battery_level_pct	10.00	100.00	74.30	16.91	−0.38	0.41	0
signal_strength_dbm	−100.00	−30.00	−65.06	12.03	0.01	0.73	0

Categorical variables (number of modalities): sensor_node_id 30, field_zone 5, crop_type 5, growth_stage 5, irrigation_status 2, sensor_status 3, target 5. None has missing values. Zones, crops, growth stages and sensors are almost uniformly distributed. sensor_status is dominated by Active (about 88 %), with Low_Battery and Weak_Signal about 12 % together.

Target distribution (development set):

Class	Count	Share
Optimal	5,121	42.8 %
Dry_Stress	2,806	23.5 %
Adverse	2,674	22.4 %
High_Humidity	873	7.3 %
Heat_Stress	478	4.0 %

The imbalance ratio is about 10.7:1, well above the 1:3 threshold, so accuracy is not a reliable metric and macro F1 and balanced accuracy will be used.

4. Visualizations

Three figures were produced on the development set (folder `plots/`): histograms with density curves for the 22 numeric variables (`numerical_hist.png`), boxplots for the 22 numeric variables (`numerical_box.png`), and bar plots for the categorical variables and the target (`categorical_barplots.png`).

Main observations:

- Most variables (soil moisture, pH, EC, N, P, K, pressure, leaf wetness, CO2, signal strength) are bell-shaped with skewness close to 0.
- Air temperature, soil temperature and evapotranspiration are bimodal. Their skewness is near 0, so the skewness statistic alone would hide this. This is consistent with a daily cycle, which Student 3's temporal analysis supports (autocorrelation of air temperature 0.84 at lag 1 and 0.85 at lag 48, i.e. 24 hours).
- Rainfall, solar radiation and light intensity are zero-inflated; wind speed and battery level are skewed; wind direction is uniform on a circular scale (0–360°).
- Several variables show small spikes at their extreme values (see Section 5.2).

5. Outlier detection

5.1 Univariate methods (IQR and z-score)

The IQR rule flags values outside [Q1 − 1.5·IQR, Q3 + 1.5·IQR]; the z-score rule flags $|z| > 3$. For 18 of the 20 retained variables (all except wind speed and rainfall), less than 1 % of values are flagged, and these sit close to the whiskers, which is the normal tail of a bell-shaped distribution. Z-scores are less reliable on the bimodal variables.

The IQR rule breaks down on zero-inflated variables. For rainfall, 85.3 % of observations are zero, so Q1 = Q3 = 0, the IQR is 0 and every positive value is flagged (14.73 %). These values are genuine rainfall events, not errors. The same holds for solar radiation and light intensity (54.2 % zeros, night time), and for irrigation and water flow (76.5 % zeros).

5.2 Values clipped at fixed bounds

Many variables have an unusually high number of rows exactly at their minimum or maximum, for example nitrogen (168 rows at 15 mg/kg), soil moisture (104 rows at 10 %), phosphorus (110 rows at 10 mg/kg), potassium (96 rows at 20 mg/kg) and battery level (999 rows at 100 %). Minimums and maximums are round numbers (15–150, 10–100, 300–600 ppm). This suggests values were truncated at predefined limits, and that the data may be synthetic or simulated. Battery at 100 % is physically natural (fully charged). This is reported as a limitation: truncation already removed the most extreme observations.

5.3 Multivariate methods (Isolation Forest and LOF)

Both methods were applied to the 20 retained numeric variables after standardization (exploration only, development set). Isolation Forest used contamination='auto' and flagged 1,037 rows (8.7 %). LOF with k = 20 was run with contamination 0.01, which forces exactly 1 % (120 rows); its count is therefore a ranking, not a finding, and is not interpreted.

Isolation Forest outliers by class:

Class	Isolation Forest outliers
Heat_Stress	33.1 %
Adverse	12.1 %
Dry_Stress	9.0 %
High_Humidity	8.4 %
Optimal	4.5 %

Outliers are concentrated in the rarest class. This is expected: every Heat_Stress row in the development set has an air temperature of at least 34.0 °C (mean 34.9 °C), against class means of 21.9 to 27.5 °C for the other classes, so these rows are far from the majority because the class is defined by high temperature. They are the signal, not noise.

6. Target leakage affecting the analysis

Student 1's audit showed that irrigation_status = On occurs only in the Dry_Stress class, and Dry_Stress never occurs with irrigation Off (Cramér's V = 1.00). `irrigation_amount_l` and `water_flow_rate_l_min` are non-zero only in the same rows. These three variables reveal the target and must be removed from the features, as the project requires.

This changed our results. In a first run that still included the two numeric irrigation variables, Isolation Forest flagged 45 % of Dry_Stress rows; after removing them, only 9.0 % are flagged and Heat_Stress becomes the class with the most outliers. The irrigation columns were therefore excluded from multivariate detection and from the Week 2 feature list. `soil_moisture_pct` was also flagged as a suspect by Student 1 (Dry_Stress always has moisture ≤ 35 %), but it is a genuine sensor measurement and is kept, pending the team and instructor decision.

7. Decisions and conclusions

Group	Variables	Decision
Less than 1 % outliers, no impossible values	16 variables	No treatment needed
Zero-inflated (IQR meaningless)	rainfall, solar radiation, light intensity	No clipping
Moderate skewed tail (3.2 %)	wind speed	Keep, or light clipping (to be tested in Week 2)
Target leakage	irrigation_amount_l, water_flow_rate_l_min (and irrigation_status)	Remove

No row will be deleted. Outliers are not errors (no physically impossible value exists), they concentrate in Heat_Stress, the smallest class (478 rows), and deleting them would worsen the class imbalance. In Week 2, outlier thresholds and scaler parameters will be fitted on training folds only inside a scikit-learn Pipeline, and models with and without outlier treatment will be compared on the shared folds using macro F1 and balanced accuracy.

8. Limitations

- The data appears truncated at fixed bounds and possibly synthetic, so extreme real-world events may be absent.
- The development set is the first 80 % in time, followed by a 24-hour gap before the test set; the test period contains slightly hotter values (maximum air temperature 39.61 °C overall versus 37.49 °C in the development set), so clipping bounds must not be too tight.
- LOF used a fixed contamination, so its output is not an independent estimate of the outlier rate. Isolation Forest and LOF were applied on standardized data fitted on the development set for exploration only; they play no role in the modelling pipeline.

9. Deliverables

`Exploratory_Data_S2_univariate.csv` , `_categorical.csv` , `_target_distribution.csv` , `_multivariate_outliers.csv` , `_outliers_by_class.csv` , `_audit.csv` , `_feature_list.csv` , and the figures in `plots/` . Code: `eda_outliers.py` (random seed 42).

PART II · WEEK 2 — OUTLIER HANDLING AND SCALING

10. Objective and components

Week 2 turns the Week 1 findings into reusable, leak-free preprocessing components: (a) an outlier handler, (b) scaling in two variants (scaled for scale-sensitive models, unscaled for tree models), and (c) an experiment that measures whether outlier treatment and scaling change the results. Everything that learns from data is fitted on the training part of each fold only, inside a scikit-learn `Pipeline` .

The code is in `outliers_scaling.py` (components) and `compare_outliers.py` (experiment). The components file reads no data files, so it can be imported without side effects.

`OutlierClipper` is a scikit-learn transformer (`fit` / `transform`). `fit()` learns clipping bounds on the training data only: IQR bounds [Q1 − factor·IQR, Q3 + factor·IQR] (default factor 1.5) or percentile bounds. `transform()` applies the stored bounds unchanged to training, validation and test data. Values beyond a bound are replaced by the bound (winsorizing). No row is ever removed, because Week 1 showed that extreme values are genuine and concentrate in the smallest class. The `columns` argument selects the columns to clip; all other columns pass through untouched.

`get_scaler(kind)` returns `StandardScaler` , `RobustScaler` or no scaler.

`make_outlier_scaler_steps(scaled, scaler_kind, clip_columns)` returns the pipeline steps in the order clipper → scaler, before the model. scaled=True is for scale-sensitive models (SVM, Logistic Regression, KNN, MLP, PCA); scaled=False is for tree models (Random Forest, Extra Trees, XGBoost, LightGBM). Bounds are computed in raw units, so clipping is applied before scaling. The constants `ZERO_INFLATED` (rainfall, solar radiation, light intensity) and `LEAKAGE_COLS` record the Week 1 decisions.

```
from outliers_scaling import make_outlier_scaler_steps
steps = make_outlier_scaler_steps(scaled=True, scaler_kind="standard")   # LR, SVM, KNN, MLP
steps = make_outlier_scaler_steps(scaled=False)                          # RF, ET, XGBoost,
LightGBM
pipe = Pipeline(steps + [("model", model)])
```

The clipper was unit-tested on a toy example: bounds learned from the training values (10 and 42) were applied unchanged to unseen values (60 became 42 and −10 became 10). RobustScaler was also checked on a column with IQR = 0 (rainfall): scikit-learn replaces the zero scale by 1, so there is no division by zero. The components were also run inside the team's assembled `preprocessing.py` , where they integrate without modification.

11. Experimental protocol

Data: the 20 numeric sensor variables of the development set (11,952 rows). These variables are identical, value by value and in row order, in the original dataset and in `data_clean.csv` (which only removes the three irrigation leakage columns), so the results do not depend on which of the two files is read. The 3,000-row test set is never used.

- **Folds:** Student 3's `splits_validation.csv` , 3 repetitions × 10 stratified folds, seed 42. In each fold the training set is the development set minus the validation rows. The same folds are used for every configuration, so comparisons are paired.
- **Models:** Logistic Regression (max_iter=1000) as the scale-sensitive model and Random Forest (100 trees) as the tree model, both with default hyperparameters. They are only probes here; optimized models belong to other binômes.
- **Metrics:** macro F1 (main) and balanced accuracy.
- **Aggregation:** one score per repetition (mean of its 10 folds), then mean ± standard deviation over the 3 repetitions. The 30 fold scores are not used for tests, because folds of the same repetition are not independent. Δ is the paired difference to the reference, repetition by repetition. A difference is practically relevant if $|\Delta F1| \geq 0.02$. Formal significance tests and confidence intervals are computed by binôme 15 from the consolidated results files.

Configurations tested (8, respecting the limit of two ablation factors per model: outlier treatment and scaling):

Model	Outlier treatment	Scaler
Logistic Regression	none (reference)	Standard
Logistic Regression	clip wind_speed_m_s only (IQR, factor 1.5)	Standard
Logistic Regression	clip all variables except the 3 zero-inflated ones (aggressive stress test)	Standard
Logistic Regression	none	none
Logistic Regression	none	Robust
Random Forest	none (reference)	none
Random Forest	clip wind_speed_m_s only	none
Random Forest	clip all except zero-inflated	none

12. Results

Mean ± standard deviation over the 3 repetitions (each the mean of 10 folds). Results from the run on Student 2's machine; the files are `results_outlier_folds.csv` , `results_outlier_summary.csv` and `results_outlier_deltas.csv` .

Model	Outlier treatment	Scaler	Macro F1	Balanced accuracy	Fit time (s)
Logistic Regression	none	Standard	0.8193 ± 0.0014	0.8272	0.61
Logistic Regression	wind only	Standard	0.8183 ± 0.0016	0.8266	0.66
Logistic Regression	all (IQR)	Standard	0.8182 ± 0.0019	0.8263	0.76
Logistic Regression	none	Robust	0.8112 ± 0.0020	0.8181	1.13
Logistic Regression	none	none	0.5450 ± 0.0027	0.5357	5.19
Random Forest	none	none	0.9976 ± 0.0005	0.9967	5.10
Random Forest	wind only	none	0.9975 ± 0.0004	0.9966	5.22
Random Forest	all (IQR)	none	0.9977 ± 0.0005	0.9967	5.11

Paired differences in macro F1 against the reference of each model, per repetition:

Model	Configuration	Δ rep 1	Δ rep 2	Δ rep 3	Mean Δ macro F1	Mean Δ balanced acc.	Relevant (≥ 0.02)
Logistic Regression	clip all, Standard	−0.0016	−0.0010	−0.0007	−0.0011	−0.0009	No
Logistic Regression	clip wind only, Standard	−0.0013	−0.0008	−0.0008	−0.0010	−0.0006	No
Logistic Regression	no scaler	−0.2750	−0.2752	−0.2726	−0.2743	−0.2916	Yes
Logistic Regression	Robust instead of Standard	−0.0089	−0.0075	−0.0078	−0.0080	−0.0092	No
Random Forest	clip all	+0.0001	+0.0001	+0.0001	+0.0001	0.0000	No
Random Forest	clip wind only	+0.0001	−0.0004	−0.0001	−0.0001	−0.0002	No

12.1 Final dataset produced with these components

The components were then applied to the whole dataset by `scale_outliers.py` , which writes `final_data_dev.csv` (11,952 development rows), `final_data_test.csv` (3,000 test rows), `final_data.csv` (both, with a `split` column) and `final_data_report.csv` . The three irrigation leakage columns are removed, the three categorical columns are one-hot encoded, and the 48 gap rows are not written. The IQR bounds (factor 1.5) and the RobustScaler statistics are learned on the development rows only and then applied unchanged to the test rows, so the test set influences nothing. The 22 numeric variables (the 20 sensor variables plus latitude and longitude) are processed: 19 are clipped and 3 (rainfall, solar radiation, light intensity) are not, as decided in Week 1. In total 1,332 values are clipped (no row removed), in 12 variables:

Variable	Lower bound	Upper bound	Values clipped	% of rows
wind_speed_m_s	-2.42	7.86	483	3.23
soil_ph	4.63	8.39	124	0.83
leaf_wetness_pct	10.08	74.49	121	0.81
signal_strength_dbm	-97.71	-32.46	108	0.72
atmospheric_pressure_hpa	990.90	1,028.98	103	0.69
soil_ec_ds_m	0.15	2.84	64	0.43
phosphorus_mg_kg	5.18	84.79	63	0.42
battery_level_pct	25.95	124.26	60	0.40
potassium_mg_kg	13.14	146.76	58	0.39
nitrogen_mg_kg	2.81	137.56	52	0.35
soil_moisture_pct	5.41	86.55	49	0.33
co2_ppm	297.26	541.84	47	0.31

No value is clipped in latitude, longitude, air_temperature_c, relative_humidity_pct, soil_temperature_c, wind_direction_deg, evapotranspiration_mm. Wind speed is the variable most affected (483 values, 3.23 %), in line with the 3.2 % IQR outliers found in Week 1; every other variable has less than 1 % of its values clipped. After scaling, the development median of each numeric variable is 0. These files are meant for the final fit and the final test; for cross-validation the same components are used inside the pipeline, fitted on each training fold, because the statistics of `final_data_dev.csv` have already seen the validation rows.

13. Interpretation and decisions

Outlier clipping has no practical effect. For both models every difference is about 0.001 or less in macro F1, roughly 20 times under the 0.02 relevance threshold, even for the aggressive configuration that clips 17 variables. For Logistic Regression clipping is slightly lower in all three repetitions (mean Δ −0.0010 and −0.0011); for Random Forest the difference is within ±0.0001. Combined with Week 1 (no impossible values, outliers concentrated in the smallest class), clipping brings no measurable benefit, but its cost is negligible as well (at most 0.0016 macro F1 in any repetition, far below the threshold). It is kept in the shared pipeline as a protective step against extreme values (IQR rule with factor 1.5, bounds fitted on the training data only, the three zero-inflated variables excluded, no row removed). Any group that prefers not to clip can set `clip=False` at no measurable cost.

Scaling is essential for scale-sensitive models. Without scaling, Logistic Regression falls from 0.8193 to 0.5450 macro F1 (Δ = −0.274, consistent across the three repetitions), and its training time is about 8.5 times higher because the optimizer fails to converge within 1,000 iterations. Variables such as `light_intensity_lux` (up to 100,000) dominate variables such as `soil_ph` (4.5–8.5). Random Forest is unaffected by scale and is therefore used unscaled, in line with the project rules.

StandardScaler and RobustScaler are close. StandardScaler is slightly better in this probe (Δ = −0.0080 for Robust, negative in all three repetitions), but the difference stays below the 0.02 relevance threshold, so the choice is not critical. The shared pipeline uses RobustScaler (median and IQR), which matches the IQR clipping rule and is less sensitive to skewed variables such as wind speed. StandardScaler remains available with `scaler="standard"` .

Decision and shared pipeline. The assembled `preprocessing.py` keeps its defaults, `clip=True` and `scaler="robust"` : scaled=True uses RobustScaler for SVM, Logistic Regression, KNN, MLP and PCA; scaled=False applies no scaling for tree models. The experiments show that `clip=False` and `scaler="standard"` give results within 0.02 macro F1 of these defaults, so groups can switch without a practical loss.

14. Observation on the target (decision pending)

Random Forest reaches macro F1 ≈ 0.998, which is unusually high. A check on the development set showed that the target behaves almost like a rule on three sensor variables: a decision tree of depth 3 gives macro F1 0.68, depth 4 gives 0.961 and depth 5 gives 0.985 (5-fold stratified cross-validation). The learned rules are: soil_moisture_pct ≤ 35 gives Dry_Stress; otherwise relative_humidity_pct > 85 gives High_Humidity; otherwise air_temperature_c separates Optimal from Adverse. In the full dataset, soil_moisture_pct ≤ 35 coincides exactly with Dry_Stress (3,557 of 3,557 rows, with no exception), which is also the set of rows where irrigation is On.

Removing soil_moisture_pct lowers a Random Forest from macro F1 0.998 to 0.673 and the Dry_Stress F1 from 1.00 to 0.02, because no other variable carries this information (its largest absolute correlation with another variable is 0.17). The irrigation variables are a consequence of the same condition and were removed as leakage. soil_moisture_pct is a genuine sensor measurement, but the target appears to be derived from it, which is what the project rule on target derivation warns about. We did not remove it and ask the instructor to decide; if it is removed, one line in the configuration of `compare_outliers.py` (`EXTRA_DROP`) is enough to rerun the experiment.

Consequence for the study: with this variable kept, strong models are close to a ceiling, so differences between models and between preprocessing choices are hard to measure.

15. Limitations of Week 2

- Only two probe models were used, with default hyperparameters; conclusions about other models rely on the general properties of scale sensitivity.
- With 3 repetitions the standard deviations are rough, and no significance test was run here (the statistical analysis is centralized by binôme 15). Conclusions rest on the size of Δ compared with the 0.02 threshold and on the consistency of its sign.
- Random Forest is close to the ceiling (≈ 0.998), so it cannot reveal small benefits of outlier treatment.
- Only the 20 numeric features were used; categorical variables were not included.
- Folds are random while the test set is chronological, with a 24-hour gap between development and test. Air temperature is strongly autocorrelated (0.85 at lag 48), so cross-validation scores may be slightly optimistic. The test period also contains hotter values (39.61 °C overall versus 37.49 °C in the development set), which is why the clipping bounds must stay wide (the IQR bounds for air temperature are 10.9 to 43.0 °C, which contain the hottest test value).
- The data appears truncated at fixed bounds and possibly synthetic, so extreme real-world events may be missing.
- Runs on different machines or library versions can differ by a few thousandths (for example in the training times and the last decimals of the scores).

16. Reproduction and deliverables

Files needed in the project: the dataset (`data_clean.csv` or the original file), `dev_index.csv` and `splits_validation.csv` (Student 3), and `Exploratory_Data_S2_feature_list.csv` (output of Week 1). `compare_outliers.py` finds them automatically; file names and the list of excluded columns are in its CONFIG block, and it stops with an explicit message if the data file is already scaled, lacks a feature column or has a different number of rows. Run `python eda_outliers.py` (about 40 seconds) then `python compare_outliers.py` (about 3 minutes). Seed 42 everywhere. Python 3.13 on the author's machine; the library versions are those of the project's shared `requirements.txt` .

Deliverables: `eda_outliers.py` , `outliers_scaling.py` , `compare_outliers.py` , `check_splits.py` (verification of Student 3's splits), the `Exploratory_Data_S2_*.csv` files, the three `results_outlier_*.csv` files, and the figures in `plots/` .

PART 3

STUDENT 3

Time-series exploration and decision gate

Temporal audit, temporal features and split generation

SUMMARY OF FINDINGS

- Timestamps are valid, unique and regular (30 minutes). Sensor, zone and position metadata are unreliable, so spatial cross-validation (GroupKFold) and out-of-group analyses across sensors and zones are removed.
- Seven variables show a temporal dependence driven by a 24-hour cycle (air and soil temperature, relative humidity, solar radiation, light, evapotranspiration, leaf wetness). Soil moisture, pressure, rainfall, NPK and wind show none. All series are stationary, with no monthly seasonality and no target drift.
- The target has a strong daily cycle (for example Heat_Stress around noon, High_Humidity at night), and no variable precedes another or the target: all relations are at lag 0.
- Decision: temporal cross-validation is feasible but not strictly necessary. The mandatory protocol is the stratified 10-fold cross-validation repeated 3 times; a TimeSeriesSplit is kept as a complementary scheme.
- Outputs: a locked test set (last 3,000 records), a development set of 11,952 records with a 24-hour gap before the test set, 30 stratified folds, 5 temporal folds and 6 temporal features (hour sine and cosine, lags 1 and 2 of air temperature and relative humidity). Measured on the shared folds, the hour encoding improves a logistic regression by +0.014 macro F1 (below the 0.02 relevance threshold), the lags add nothing, and a random forest is unaffected.

1. Scope and data used

This section reports the temporal audit (Week 1) and the temporal features and split generation (Week 2). The dataset has 15,000 records and 32 columns, from 2024-01-01 00:00 to 2024-11-08 11:30. All numerical results are stored in `results/Exploratory_temporal.csv` (148 rows) and all figures in `figures/`. Analyses use the global series ordered by timestamp. Monthly analyses use January to October only, because November contains only 8 days of data.

The target is `environmental_condition` (five classes): Optimal 42.6%, Dry_Stress 23.7%, Adverse 22.4%, High_Humidity 7.3% and Heat_Stress 4.0%. The imbalance ratio between the largest and the smallest class is about 10.6:1.

2. Timestamp quality

Check	Result
Invalid or missing timestamps	0
Duplicated timestamps	0
Sampling interval	30 minutes for 100% of consecutive records (minimum = maximum)

Check	Result
Chronological order	Records already sorted
Coverage	312 days (15,000 records x 30 min = 312.5 days)

Timestamps are valid, unique and perfectly regular. Temporal analysis and temporal validation are therefore technically feasible.

3. Reliability of sensor, zone and position metadata

The dataset contains 30 sensors (453 to 553 records each) and 5 zones (2,924 to 3,055 records each). Each of the 30 sensors appears in all 5 zones, with about 100 records per zone. Sensor SN_001, for example, has 501 records and 500 distinct latitude values, i.e. a new position at almost every record. Since all 15,000 timestamps are unique, exactly one sensor reports at each 30-minute step.

Consequently, `sensor_node_id`, `field_zone` and the coordinates do not describe stable physical locations. They are not reliable as grouping variables. Following the project protocol, spatial cross-validation (GroupKFold) and out-of-group generalization analyses are removed from the study design, and this is reported in the limitations.

4. Temporal dependence and stationarity

The ADF test indicates that all 20 continuous variables tested are stationary ($p < 0.001$), so there is no long-term trend. The Ljung-Box test (lag 48, i.e. 24 hours) shows temporal dependence for seven variables ($p < 0.001$), all driven by the daily cycle. The partial autocorrelation (PACF) shows a short direct memory (lags 1 and 2).

Variable	Ljung-Box p	PACF lag 1	PACF lag 2
air_temperature_c	< 0.001	0.837	0.403
soil_temperature_c	< 0.001	0.777	0.398
relative_humidity_pct	< 0.001	0.682	0.378
solar_radiation_w_m2	< 0.001	0.921	0.306
light_intensity_lux	< 0.001	0.902	0.336
evapotranspiration_mm	< 0.001	0.936	0.308
leaf_wetness_pct	< 0.001	0.279	0.210
nitrogen_mg_kg	0.025	0.005	-0.002

Nitrogen is not considered temporally dependent: its PACF is close to zero at all lags, and with 20 tests performed one false positive at the 5% level is expected. The remaining 12 variables show no significant dependence (for example soil_moisture_pct $p = 0.397$, atmospheric_pressure_hpa $p = 0.534$, rainfall_mm $p = 0.455$; co2_ppm is borderline at $p = 0.051$).

For air temperature, the autocorrelation is 0.837 at lag 1 and 0.848 at lag 48, and about -0.85 near lag 24 (read from the plot), which reveals a 24-hour cycle. For soil moisture, the lag-1 autocorrelation is -0.005 and the ACF/PACF behave like white noise (Figures 1 and 2).

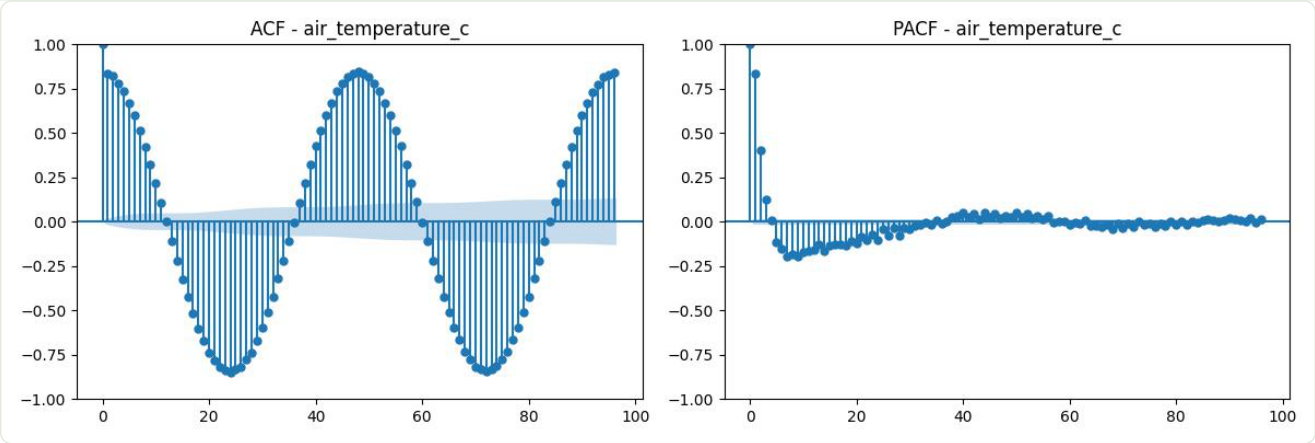


Figure 1. ACF (left) and PACF (right) of air_temperature_c, lags 0 to 96 (two days). The ACF oscillates with a period of 48 steps (24 h); the PACF is large only at lags 1 to 3.

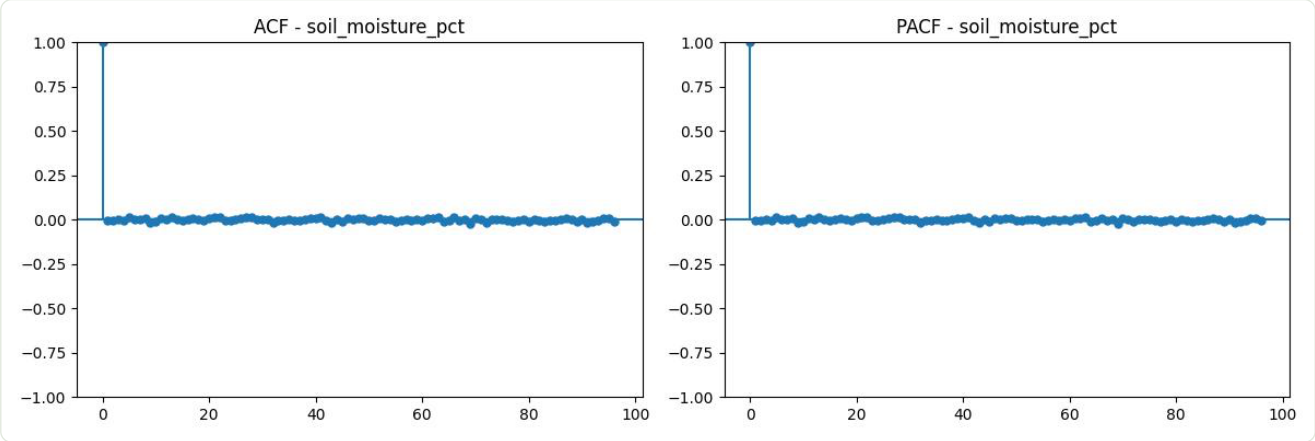


Figure 2. ACF (left) and PACF (right) of soil_moisture_pct: all bars stay close to zero at every lag (white noise).

5. Daily and seasonal patterns

Daily cycle: the mean air temperature varies from 20.8 C at 00:00 to 33.1 C at 12:00. Soil moisture shows no systematic daily pattern.

Monthly pattern (January to October): monthly means are almost constant, and the Kruskal-Wallis test does not detect any month effect (Figure 3).

Variable	Relative range of monthly means	Kruskal-Wallis p
air_temperature_c	0.5%	0.9997
soil_temperature_c	0.5%	0.9998
relative_humidity_pct	1.0%	0.931
soil_moisture_pct	2.8%	0.387
solar_radiation_w_m2	2.0%	1.000

There is a strong daily cycle and no monthly seasonality. With only 10 months of data, an annual seasonality cannot be assessed.

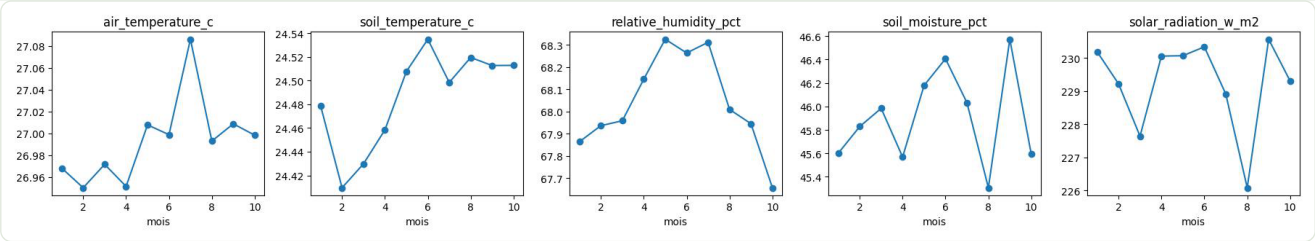


Figure 3. Monthly means of five variables, January to October. The vertical axes are zoomed: the largest variation is below 3% of the mean.

6. Target: drift and daily cycle

Target drift. The class distribution is stable across months (chi-square test month x class, January to October: p = 0.526). Monthly proportions stay within these ranges: Adverse 0.205 to 0.237, Dry_Stress 0.215 to 0.258, Heat_Stress 0.032 to 0.047, High_Humidity 0.067 to 0.077, Optimal 0.401 to 0.442. No target drift is detected (Figure 4).

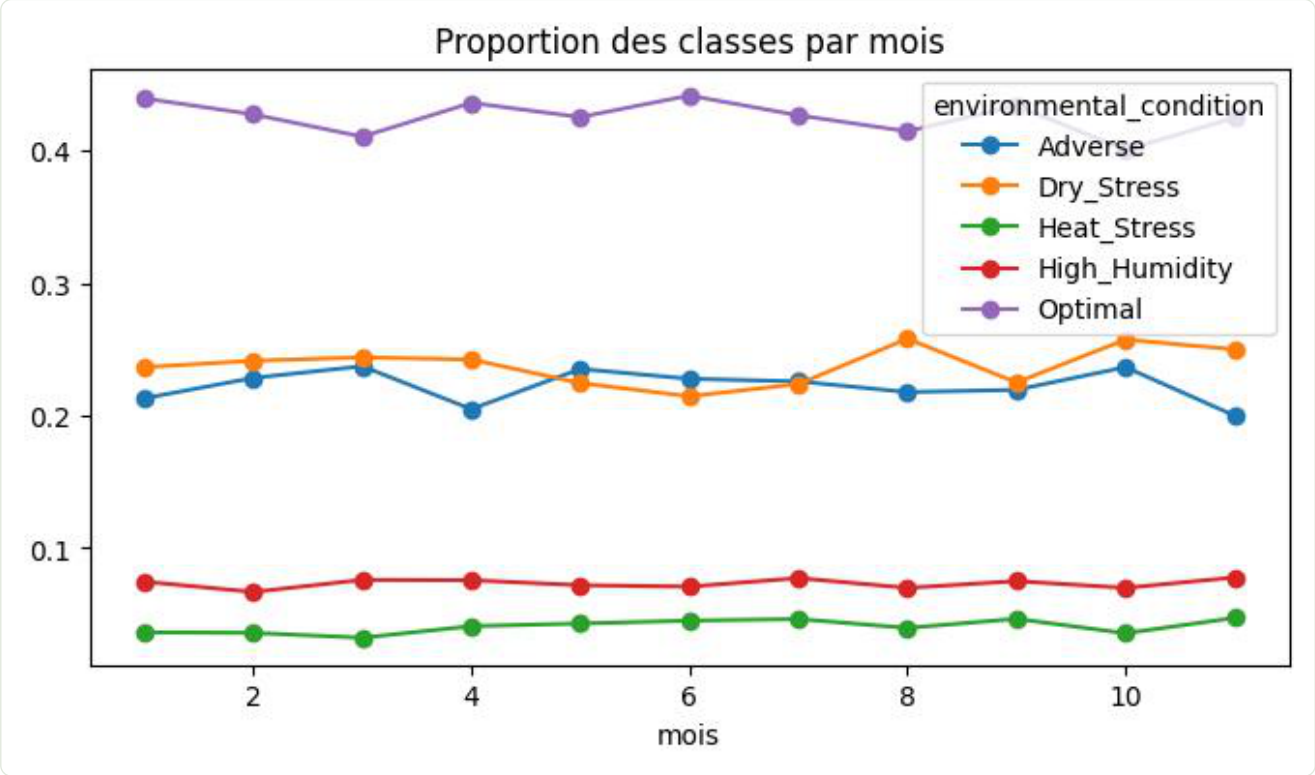


Figure 4. Monthly proportion of each class (November contains only 8 days of data).

Daily cycle of the target. The class distribution depends strongly on the hour of the day (chi-square test hour x class: p < 0.001, Figure 5):

- Heat_Stress is essentially absent outside 08:00 to 16:00 and peaks at 12:00 (22.9%).
- High_Humidity is essentially absent between 06:00 and 18:00 and peaks at 23:00 (28.2%).
- Optimal peaks at 06:00 to 07:00 (about 70%) and at 17:00 (73.1%).
- Adverse peaks between 10:00 and 13:00 (up to 37.2%).
- Dry_Stress stays between 19.7% and 26.9% at all hours.

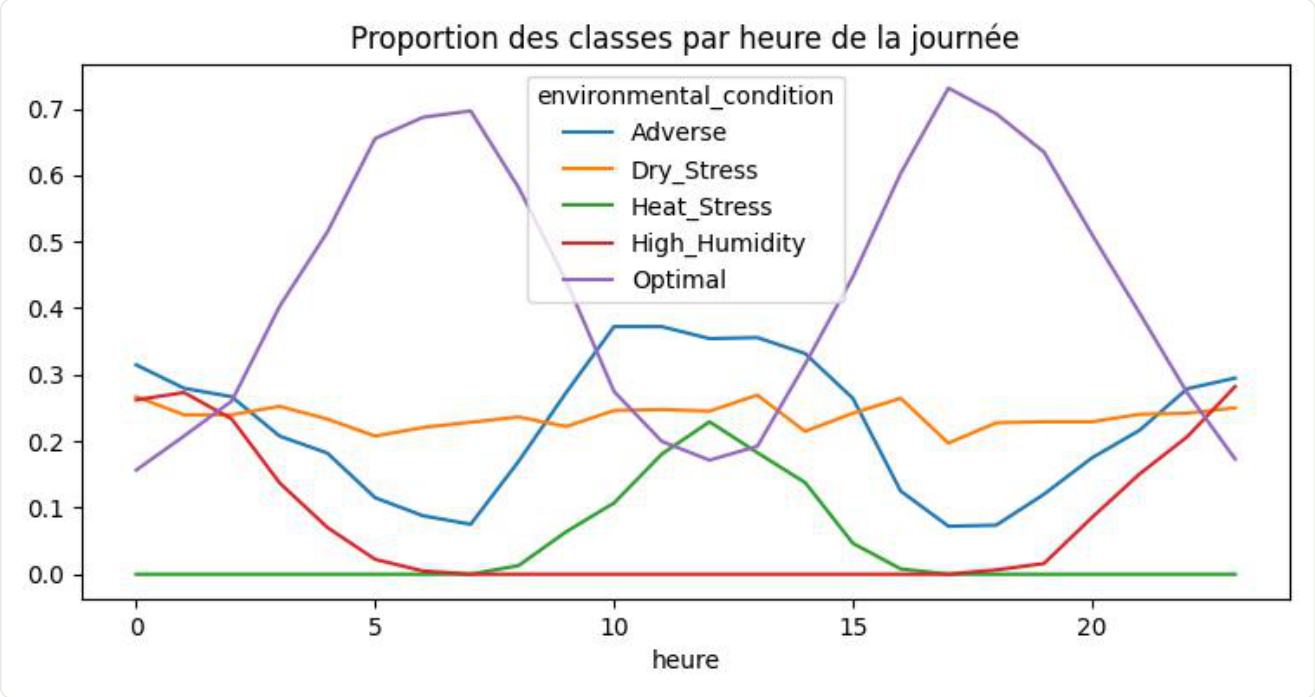


Figure 5. Proportion of each class by hour of the day: a strong daily cycle for Heat_Stress, High_Humidity, Optimal and Adverse; Dry_Stress stays flat.

The probability that two consecutive records have the same class is 0.358, against 0.294 expected under independence. This weak persistence is explained by the daily cycle: if the classes were independent given the hour of the day, the expected probability would be about 0.358, which equals the observed value. There is no evidence of a memory between classes beyond this cycle.

7. Cross-correlation and lead-lag relationships

Within a window of plus or minus 6 hours (12 steps), the best lag is 0 for every pair tested, and the correlation at the best lag equals the correlation at lag 0.

Pair (x to y)	Best lag	Correlation
solar_radiation_w_m2 to air_temperature_c	0	0.819
air_temperature_c to relative_humidity_pct	0	-0.766
air_temperature_c to soil_temperature_c	0	0.961
relative_humidity_pct to leaf_wetness_pct	0	0.637

An initial search over plus or minus 24 hours returned lags of +24 and -24 with a reversed sign. This is an artefact of the 24-hour periodicity (correlation alternates between positive and negative every 12 hours) and not a genuine lead-lag; the search was therefore restricted to 6 hours. For the target (search over plus or minus 24 hours), the best lag is also 0: air temperature versus the Heat_Stress indicator (r = 0.352) and relative humidity versus the High_Humidity indicator (r = 0.471). No variable precedes another variable or the target; the class is determined by the conditions at the same instant.

8. Decision gate: is temporal cross-validation appropriate?

Criterion	Evidence	Verdict
Timestamp quality	Regular 30-min grid, no duplicates, sorted	Supports feasibility
Temporal dependence	Strong for 7 variables (daily cycle), none for 12 others; weak for the target	Partial
Non-stationarity or drift	All series stationary, no target drift, no monthly seasonality	Does not require it
Intended generalization	Classification of the current state from simultaneous sensor readings, not forecasting	Does not require it
Spatial metadata	Unreliable	GroupKFold removed

Decision. Temporal validation is feasible but not strictly necessary. The mandatory protocol is the stratified 10-fold cross-validation repeated 3 times. A TimeSeriesSplit is kept as a complementary scheme to check robustness across periods, because consecutive records are strongly correlated (autocorrelation 0.84 at lag 1 for air temperature), which could inflate the scores of random splits. Spatial cross-validation and out-of-group generalization across sensors and zones are removed; robustness across time periods is assessed with the TimeSeriesSplit.

9. Temporal feature engineering

Features are computed in `src/temporal_features.py` using only past observations (positive shifts) on the global series, and none uses the target:

- `heure_sin` and `heure_cos`: cyclical encoding of the time of day (justified by the daily cycle of the features and of the target).
- lag 1 and lag 2 (30 and 60 minutes) of `air_temperature_c` and `relative_humidity_pct`: justified by their strong PACF at lags 1 and 2; they drive the Heat_Stress and High_Humidity classes.
- No temporal features for variables without temporal dependence (soil moisture, pressure, rainfall, NPK, wind, battery, signal).

The first record has a missing lag 1 and the first two records have a missing lag 2; these values are left to the imputation step of the pipeline. Other dependent variables (soil temperature, radiation, light, evapotranspiration, leaf wetness) are strongly correlated with the two selected ones and remain candidates for the feature selection step.

Rolling statistics and differences were not built: the PACF shows a direct memory limited to the first two lags, which the lag features already capture, and the daily cycle is encoded by sine and cosine. No time-since-event feature applies, since rainfall shows no temporal dependence and the irrigation variables are removed as target leakage. The project guidelines recommend computing features per sensor node; this was not possible here because the sensor identifier does not correspond to a stable location and each sensor reports only about once every 10 hours, so the lags are computed on the global series ordered by timestamp (see Limitations). The temporal features file is indexed by the original row number, so that it can be joined to the development set.

Evaluation of the temporal features. Their effect was measured with the shared preprocessing pipeline on the 30 shared stratified folds, using a logistic regression (scaled) and a random forest (unscaled) with default hyperparameters. Scores are one value per repetition (mean of its 10 folds), then mean and standard deviation over the 3 repetitions; the baseline logistic regression (0.808) matches the value reported for the same pipeline in the feature analysis section.

Model	Features	Macro F1	Balanced accuracy	Delta macro F1 vs base (repetitions 1, 2, 3)
Logistic regression	base (no temporal feature)	0.8076 +/- 0.0009	0.8151	reference
Logistic regression	+ hour sine and cosine	0.8217 +/- 0.0020	0.8240	+0.0127, +0.0148, +0.0146
Logistic regression	+ hour and 4 lags (6 features)	0.8192 +/- 0.0013	0.8214	+0.0111, +0.0125, +0.0110
Random forest	base	0.9960 +/- 0.0009	0.9941	reference
Random forest	+ 6 temporal features	0.9953 +/- 0.0004	0.9933	-0.0018, -0.0006, +0.0003

For the logistic regression, the hour encoding gives a consistent gain (positive in the three repetitions, mean +0.014) that stays below the practical relevance threshold of 0.02. The lags add nothing beyond the hour encoding (the six features give +0.012, slightly less than the hour encoding alone). For the random forest the difference is within noise (mean -0.0007, mixed signs), because the target is almost a rule on instantaneous readings and the model is close to its ceiling. Conclusion: the temporal features are not necessary for tree models; the hour encoding is a cheap optional addition for linear models; the lags are not retained as a default. No significance test was run (these tests are centralized by binome 15) and the models are probes with default hyperparameters.

10. Splitting protocol

- Locked test set: the last 3,000 records (2024-09-07 00:00 to 2024-11-08 11:30), used only for the final evaluation.
- Development set: 11,952 records (2024-01-01 00:00 to 2024-09-05 23:30). The 48 records (24 hours) between the development set and the test set are discarded to avoid contiguity between training and test data.
- Class proportions in development and test sets are close (computed on the initial split before the 24-hour gap: Optimal 0.428 vs 0.415, Heat_Stress 0.040 vs 0.041).
- Main scheme: RepeatedStratifiedKFold, 10 folds x 3 repetitions, `random_state = 42` (about 1,195 validation records per fold). Validation indices are saved to disk and shared by all groups.
- Complementary scheme: TimeSeriesSplit with 5 folds and a gap of 48 steps (24 hours); each validation fold has 1,992 records and the training size grows from 1,944 to 9,912 records. This scheme is not stratified.

11. Limitations

1. Temporal features are computed on the global series because sensor metadata are unreliable. The protocol asks for per-sensor features; this deviation is a consequence of the data and consecutive records come from different sensors and zones.
2. The data cover only 10 months (November is partial), so annual seasonality and generalization to other seasons cannot be evaluated. The test set covers only September to November.
3. Temporal dependence tests use a single lag (48), 20 tests were run without multiple-testing correction, and ADF p-values are reported as $p < 0.001$.
4. Several patterns suggest that the data may be simulated (month-to-month p-values close to 1, lead-lag relations exactly at lag 0, sensor positions changing at every record). Conclusions may not transfer to real deployments.
5. Spatial cross-validation and out-of-group generalization across sensors and zones were not performed because the metadata are unreliable. A validation by groups of periods (for example leave-one-month-out) was not performed either; robustness across time periods relies on the TimeSeriesSplit.
6. The temporal cross-validation is not stratified, and its first fold is trained on only 1,944 records, so its scores are expected to be lower and more variable.
7. Lag features were built for two variables only; other temporally dependent variables are left to feature selection.
8. Potential target leakage from some variables (for example the irrigation variables) is handled in the dataset audit and not in this section.

Note on the exploratory scope. The exploratory temporal statistics were computed on the whole series, including the period later used as the locked test set. They are descriptive and no model parameter is learned from them; the temporal features depend only on timestamps and past feature values.

12. Reproducibility

All stochastic operations use `random_state = 42`. Running `make_splits.py` twice produced identical files. Deliverables of this part: `results/Exploratory_temporal.csv`, `figures/` (five figures), `src/make_splits.py` (outputs `dev_index.csv`, `test_index.csv`, `splits_validation.csv`, `splits_temporel.csv` in `results/splits/`) and `src/temporal_features.py` (output `results/temporal_features.csv`). The `index` columns refer to the row index of the original CSV file: do not sort, filter or drop rows before using them.

To reproduce this part, run the following commands from the repository root:

```
python src/make_splits.py data/precision_agriculture_dataset.csv
python src/temporal_features.py data/precision_agriculture_dataset.csv
```

PART 4

STUDENT 4

Feature analysis, selection and leakage control

Feature relevance, selection, class imbalance, time-series robustness and leakage tests

MAIN FINDINGS

- **Three irrigation columns reveal the target** (irrigation is On exactly when the class is Dry_Stress) and are removed: this is leakage, not signal.
- **The target is a threshold rule** on `soil_moisture_pct` (Dry_Stress ⇔ ≤ 35), `relative_humidity_pct` and `air_temperature_c`. A depth-6 decision tree reaches macro F1 0.995.
- **Three core variables carry the signal; 5 to 6 features give the best logistic regression score.** ANOVA, RFE and L1 all keep the three variables of the rule, and the best macro F1 (about 0.817) is reached with 5 to 6 features, against 0.809 with all 33.
- **Rebalancing helps a linear model, not a forest.** SMOTE and ADASYN raise the balanced accuracy of logistic regression from 0.816 to about 0.847; for random forest no strategy beats the baseline (macro F1 0.996).
- **Automatic tests** check that nothing from validation or test rows reaches training (16 tests pass).

1. Data and splits

The 15,000 rows are sensor readings every 30 minutes, ordered in time. The split is chronological, defined once by the team's `make_splits.py` (seed 42), and every analysis here uses the development set only.

- **Development set:** first 11,952 rows (2024-01-01 to 2024-09-05).
- **Gap:** the next 48 rows (24 hours) are dropped, so the first test rows are not near-copies of the last training rows.
- **Locked test set:** last 3,000 rows (2024-09-07 to 2024-11-08), used once for the final evaluation.
- **Validation:** 3 × 10 stratified folds on the development set (30 folds), plus 5 time-series folds with a 24-hour gap.

Class distribution is similar in both parts (development / test): Optimal 42.8 % / 41.5 %, Dry_Stress 23.5 % / 24.4 %, Adverse 22.4 % / 22.7 %, High_Humidity 7.3 % / 7.3 %, Heat_Stress 4.0 % / 4.1 %. The ratio between the largest and the smallest class is 10.7 : 1, above the project limit of 3 : 1, so imbalance has to be handled (section 5). Because of it, accuracy is not used: 43 % of rows are Optimal. All models are judged by macro F1, balanced accuracy and per-class recall.

2. Target leakage

Crossing `irrigation_status` with the target on the development set shows that irrigation is **On** in all 2,806 Dry_Stress rows and in none of the others, and that `irrigation_amount_l` and `water_flow_rate_l_min` are above zero only when irrigation is On. These three columns are the consequence of the label, not a measurement available beforehand, so a model using them would score perfectly and be useless in practice. They are removed once, as a fixed rule (not learned from data).

Five more columns are kept out of the features: `timestamp` (used for ordering and splitting, not as a variable), `sensor_node_id` and `field_zone` (identifiers that would not generalise to new sensors; Cramér's V with the target 0.049 and 0.019), and `latitude` and `longitude` (every sensor appears in all zones and its position changes at almost every record, so the coordinates describe no stable location and carry no signal).

3. Feature analysis

3.1 The target follows a rule

A decision tree of increasing depth, cross-validated, recovers the labels almost exactly: macro F1 0.553 at depth 2, 0.962 at depth 4, 0.995 at depth 6 and 0.999 at depth 8. The only features with non-zero importance are `soil_moisture_pct` (0.444), `relative_humidity_pct` (0.293) and `air_temperature_c` (0.263). This explains why tree models are expected to reach very high scores, and why `soil_moisture_pct` is flagged for a decision by the course teacher: it is a legitimate sensor reading, but it nearly defines the class Dry_Stress.

Feature	ANOVA F	Mutual info	VIF (20 numeric)
soil_moisture_pct	3,563	0.590	1.0
air_temperature_c	785	0.397	16,401.6
relative_humidity_pct	1,046	0.396	4,231.3
evapotranspiration_mm	705	0.252	214,395.9
soil_temperature_c	701	0.223	13.1
solar_radiation_w_m2	552	0.119	88,599.3
light_intensity_lux	535	0.107	47.9
leaf_wetness_pct	364	0.071	2.2

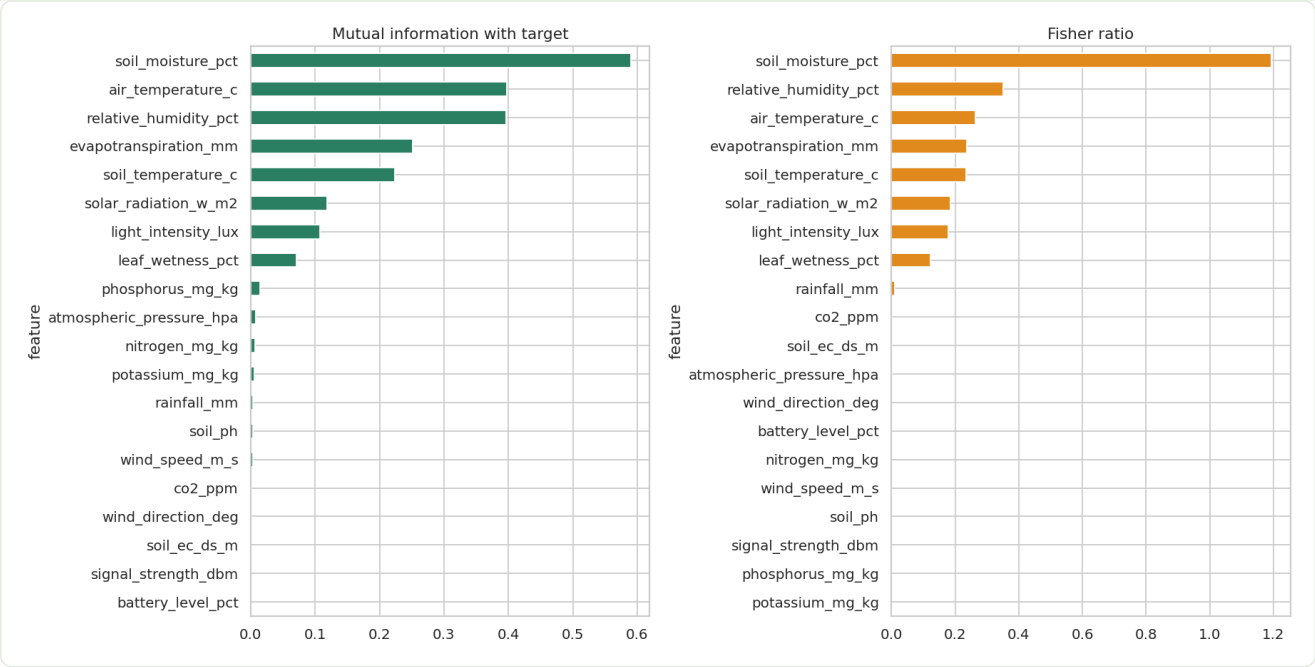


Figure 1. Univariate relevance of each feature (ANOVA F, mutual information) on the development set.

3.2 Correlations and multicollinearity

Five pairs of numeric variables have a Pearson correlation above 0.9: `solar_radiation_w_m2` with `light_intensity_lux` (0.989) and with `evapotranspiration_mm` (0.973), `light_intensity_lux` with `evapotranspiration_mm` (0.963), `air_temperature_c` with `soil_temperature_c` (0.961) and with `evapotranspiration_mm` (0.910). Spearman gives the same picture. The variance inflation factor (VIF) confirms it: `evapotranspiration_mm` and `solar_radiation_w_m2` are almost perfectly explained by the others.

An iterative elimination removes the variable with the highest VIF while the maximum stays above 10, but **never removes a variable the target rule depends on** . Result: `evapotranspiration_mm` , `solar_radiation_w_m2` and `soil_temperature_c` are flagged redundant. Seven more numeric variables (soil pH, wind speed, CO_2, wind direction, soil EC, battery level, signal strength) have no detectable relation to the target (ANOVA $p > 0.05$ and mutual information < 0.005). These flags are recommendations; the dataset keeps all 23 variables and section 4 tests them with models.

Figure 2. Pearson and Spearman correlation heatmaps of the numeric features.

3.3 Separability of the classes

PCA on the 20 standardised numeric features shows no low-dimensional structure: the first component explains 27.9 % of the variance, and 11, 14 and 15 components are needed for 80 %, 90 % and 95 %. t-SNE and UMAP show overlapping clouds. The overall silhouette score of the true labels is −0.04 (0.013 on the three signal features), with Heat_Stress (0.20) and High_Humidity (0.15) the only classes with positive cohesion. In other words the classes are **not separated by distance** : they are separated by thresholds, which is why distance-based methods (KNN, SVM with RBF kernel) are expected to do worse than trees.

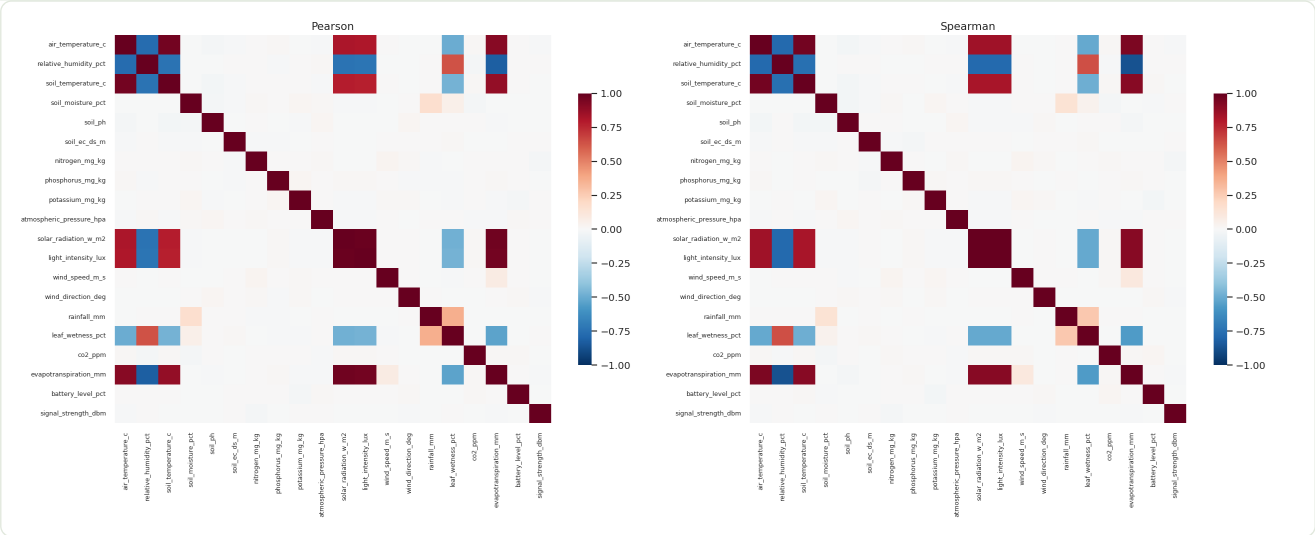


Figure 3. PCA projection of the development set coloured by class.

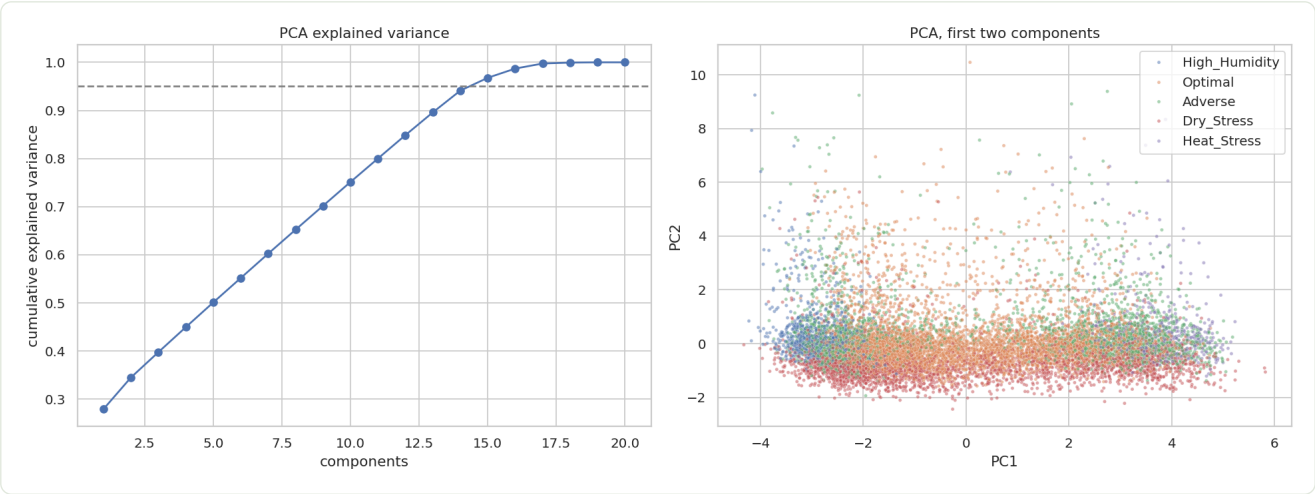


Figure 4. t-SNE and UMAP projections: classes overlap heavily.

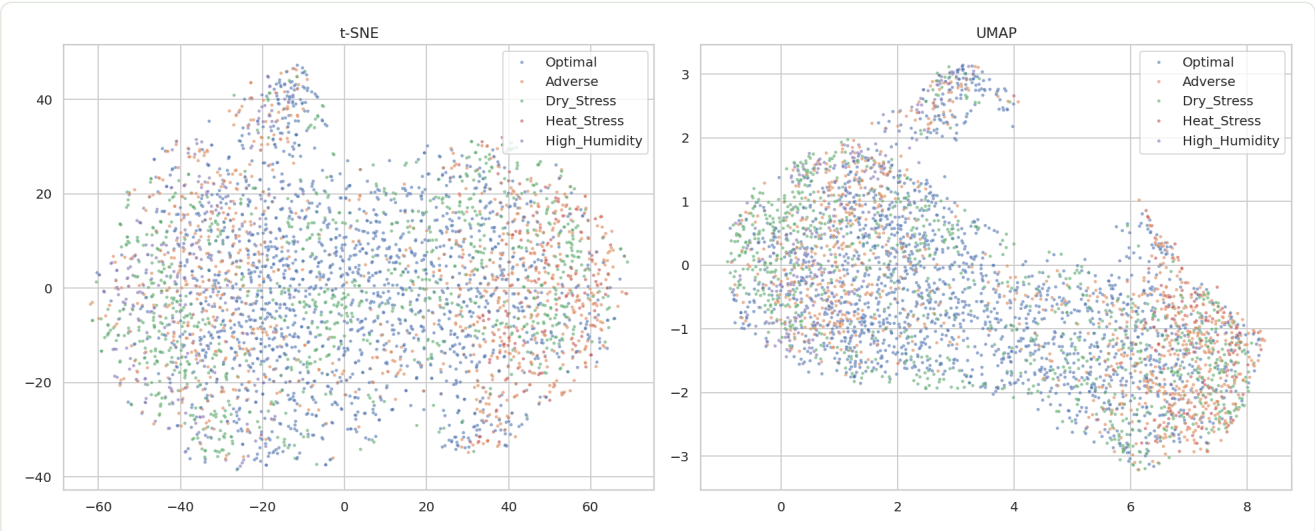


Figure 5. Distance between class centroids.

4. Feature selection

Three families of selectors are compared, all fitted on the **training part of each of the 30 folds** and applied after the preprocessing (one-hot columns included, 33 features): **ANOVA** (filter, `SelectKBest`), **RFE** with logistic regression (wrapper) and **L1** (embedded, one-vs-rest logistic regression; a variable is kept if any class gives it a non-zero weight). For each method the number of features k (or the L1 strength C) is chosen as the **smallest value within 0.005 of the best macro F1** , which favours parsimony.

Method	Setting chosen	Mean features kept	Macro F1 (30 folds)
ANOVA (filter)	k = 5	5.0	0.811 ± 0.010
RFE (wrapper)	k = 5	5.0	0.815 ± 0.010
L1 (embedded)	C = 0.003	6.3	0.817 ± 0.009
All 33 features	—	33.0	0.809 ± 0.010

The three methods agree on the core: `air_temperature_c` , `relative_humidity_pct` and `soil_moisture_pct` are kept in every fold by all of them, which matches the target rule found in section 3.1. Evapotranspiration and solar radiation are kept by two methods, the rest by at most one.

Feature	ANOVA	RFE	L1	Methods keeping it
air_temperature_c	1.00	1.00	1.00	3
relative_humidity_pct	1.00	1.00	1.00	3
soil_moisture_pct	1.00	1.00	1.00	3
evapotranspiration_mm	1.00	1.00	0.00	2
solar_radiation_w_m2	0.00	0.63	1.00	2
soil_temperature_c	1.00	0.37	0.30	1
light_intensity_lux	0.00	0.00	1.00	1
rainfall_mm	0.00	0.00	1.00	1

Selection frequency over the 30 folds at the chosen setting (1.00 = kept in every fold).

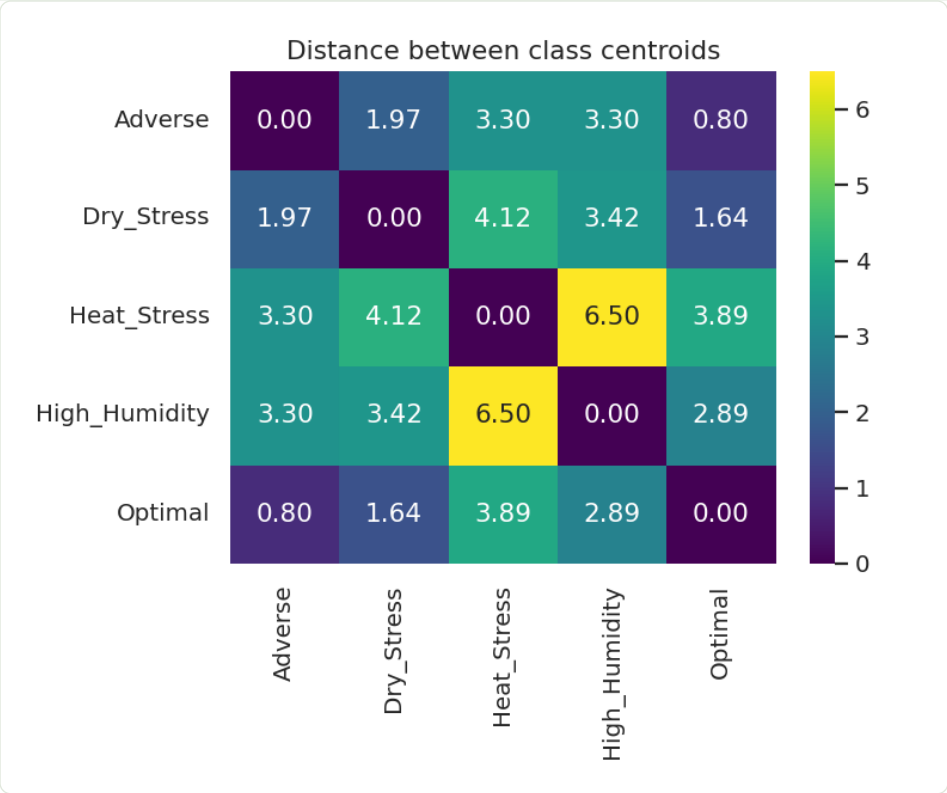


Figure 6. Macro F1 against number of features kept (mean ± standard error over 30 folds).

Reading the result honestly. The gain over using all 33 features is small (+0.003 to +0.008 macro F1, with a fold standard deviation of about 0.01), so the main benefit of selecting 5 to 6 features is a simpler, faster and more stable model, not a large accuracy gain. Selection is available to the other groups as `build_selector(method, value)` and plugs into the pipeline; it is not applied to the shared dataset.

5. Class imbalance

Four strategies are compared with a baseline, for logistic regression (scaled) and random forest (unscaled), on the same 30 folds. Each strategy lives **inside the pipeline** , so synthetic or removed rows exist only in the training part of a fold; validation rows are never resampled. The strategies are class weights, SMOTE, ADASYN and random undersampling.

Model	Strategy	Macro F1	Balanced acc.	Δ bal. acc. vs none (± s.e.)
Logistic regression	none (baseline)	0.809	0.816	—
Logistic regression	class weights	0.784	0.840	+0.024 ± 0.002
Logistic regression	SMOTE	0.803	0.847	+0.031 ± 0.001
Logistic regression	ADASYN	0.803	0.847	+0.031 ± 0.001
Logistic regression	undersampling	0.737	0.817	+0.001 ± 0.001
Random forest	none (baseline)	0.996	0.994	—
Random forest	class weights	0.996	0.995	+0.000 ± 0.000
Random forest	SMOTE	0.994	0.994	-0.000 ± 0.000
Random forest	ADASYN	0.994	0.994	-0.000 ± 0.000

Model	Strategy	Macro F1	Balanced acc.	Δ bal. acc. vs none (± s.e.)
Random forest	undersampling	0.974	0.978	-0.016 ± 0.001

For **logistic regression** , SMOTE, ADASYN and class weights raise balanced accuracy by 0.024 to 0.031 (a paired difference of more than 15 standard errors) while macro F1 stays level (0.80 for SMOTE and ADASYN against 0.81 for the baseline) and drops for undersampling and class weights. The gain comes from the minority and hard classes, at a cost for the majority class:

Strategy (logistic regression)	Adverse	Dry_Stress	Heat_Stress	High_Humidity	Optimal
none (baseline)	0.35	0.98	0.91	0.97	0.87
class weights	0.55	0.97	1.0	0.99	0.69
SMOTE	0.58	0.97	1.0	0.99	0.69
ADASYN	0.59	0.97	1.0	0.99	0.68
undersampling	0.48	0.96	1.0	0.99	0.66

Adverse recall rises from 0.35 to 0.59 and Heat_Stress from 0.91 to 0.997, while Optimal falls from 0.87 to 0.69. For **random forest** , the baseline is already the best (macro F1 0.996, balanced accuracy 0.994); no strategy changes the scores (differences within ±0.0005), except undersampling, which loses 0.016 of

balanced accuracy. Recommendation: use SMOTE or ADASYN for linear models when minority recall matters, and no rebalancing for tree ensembles.

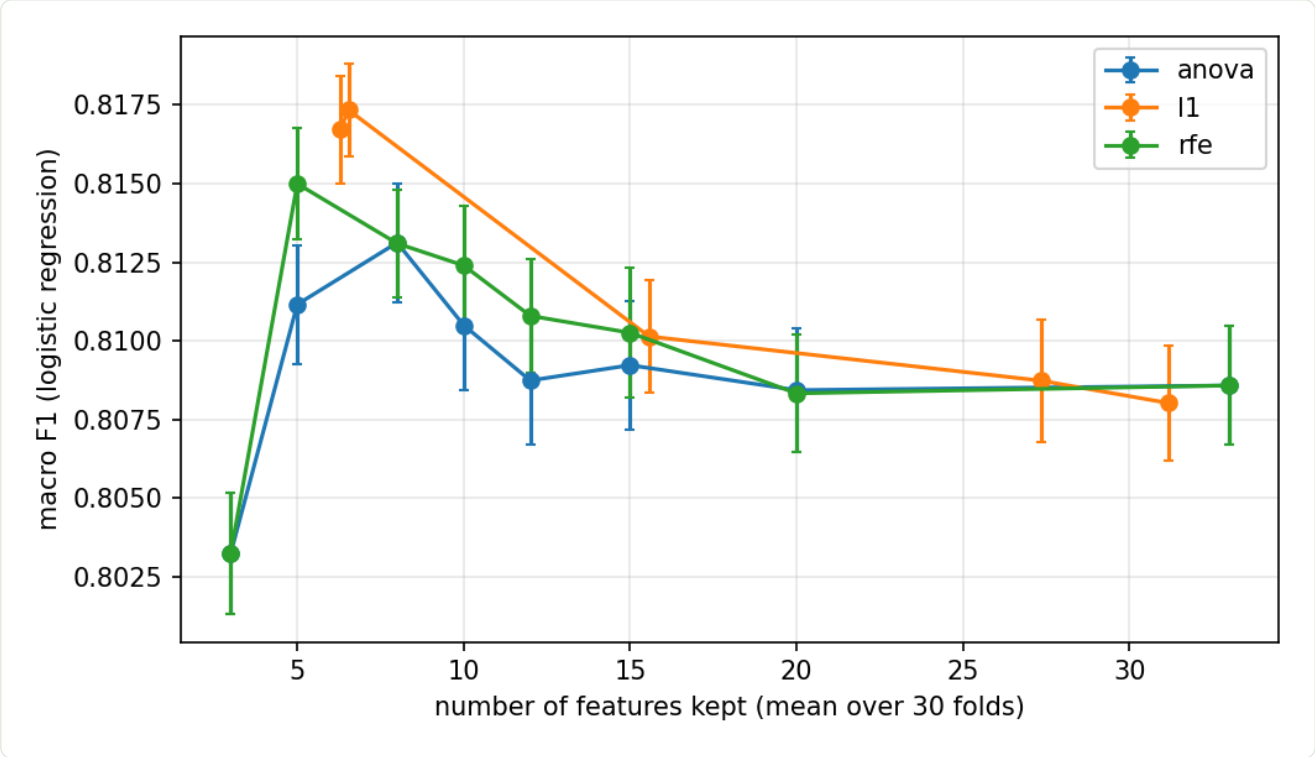


Figure 7. Macro F1 and balanced accuracy for each strategy and model (mean ± standard error over 30 folds).

Caveat: SMOTE and ADASYN run after one-hot encoding, so synthetic rows can contain fractional values in the one-hot columns. This is acceptable for a comparison but should be kept in mind.

6. Robustness on the time-series folds

Both comparisons were repeated on the 5 time-series folds of the team's temporal split (training on the past, a 24-hour gap, 1,992 validation rows per fold, not stratified; `--folds temporal` in the two scripts). The conclusions hold, with lower and more variable scores because the first folds train on little data. ANOVA, RFE and L1 again keep the three core variables in every fold, and the smallest set within tolerance is the 3-variable one (macro F1 0.798, against 0.778 with all 33). SMOTE and ADASYN raise the balanced accuracy of logistic regression from 0.778 to 0.830 and 0.832 (+0.05, about 9 standard errors), while for random forest the baseline is the best (macro F1 0.992) and undersampling is clearly worse (balanced accuracy −0.037). The choice of features and of the rebalancing strategy therefore does not depend on the way the folds are cut.

7. Leakage control and tests

The rule of the pipeline is simple: **anything learned from data is fitted on the training part only**. Fixed rules (dropping the leakage columns) are applied once. The median imputer, IQR clipping bounds, robust scaler, one-hot encoder, selector and sampler all live inside one `imblearn` pipeline. Ten automatic tests (`tests/test_leakage.py`) check it:

- the target and the three irrigation columns are not among the features;
- validation folds are disjoint from their training rows and together cover the development set exactly;
- the test set is the chronological end of the data and does not overlap the development set;
- the scaler's centre and scale equal the statistics of the training rows, not of the whole dataset;
- multiplying the validation rows by 1,000 changes nothing in the fitted pipeline;
- with shuffled labels, scores fall to chance level (the pipeline cannot learn what is not there);
- running the pipeline twice gives identical predictions.

Six more tests cover the dataset-level scaling script (test rows do not change the learned bounds, zero-inflated columns are not clipped, the dev/test cut matches the saved splits, including the 48-row gap). All 16 tests pass.

8. Deliverables and how to use them

Datasets (`python src/preprocessing.py`, needs the saved splits): `precision_agriculture_cleaned_{dev,test}.csv` (leakage and identifier columns removed, values untouched) and `precision_agriculture_preprocessed_{dev,test}.csv` (imputed, clipped, scaled, one-hot, fitted on the development rows only): 11,952 development rows, 3,000 test rows, 33 features.

Pipeline (`src/preprocessing.py`): `load_dev_and_folds` gives the development data and the 30 folds, `build_pipeline(model, X, scale, selector, sampler)` gives the full leakage-free pipeline. For model comparison and tuning, use the pipeline with the 30 folds: the pre-scaled CSV files are for the final fit and the single final test, because cross-validating on them would let the scaler see validation rows.

9. Decisions pending and limitations

- `soil_moisture_pct`: kept for now. Removing it (one line in `LEAKAGE_COLS`) would make the task much harder and the scores more informative; this needs the teacher's answer.

- **Identifier columns** (`timestamp`, `sensor_node_id`, `field_zone`) are excluded; temporal features are left to the time-series part of the project.
- **Selection and VIF** are computed on one linear model and on linear correlations; a tree-based selector could rank variables differently.
- **Scores are high** for tree models because the target is a rule. They say little about performance on real, noisier labels.
- **Chronological test set:** the test period (September to November) may differ in season from the development period, so a drop in test scores is expected and is not a sign of a bug.